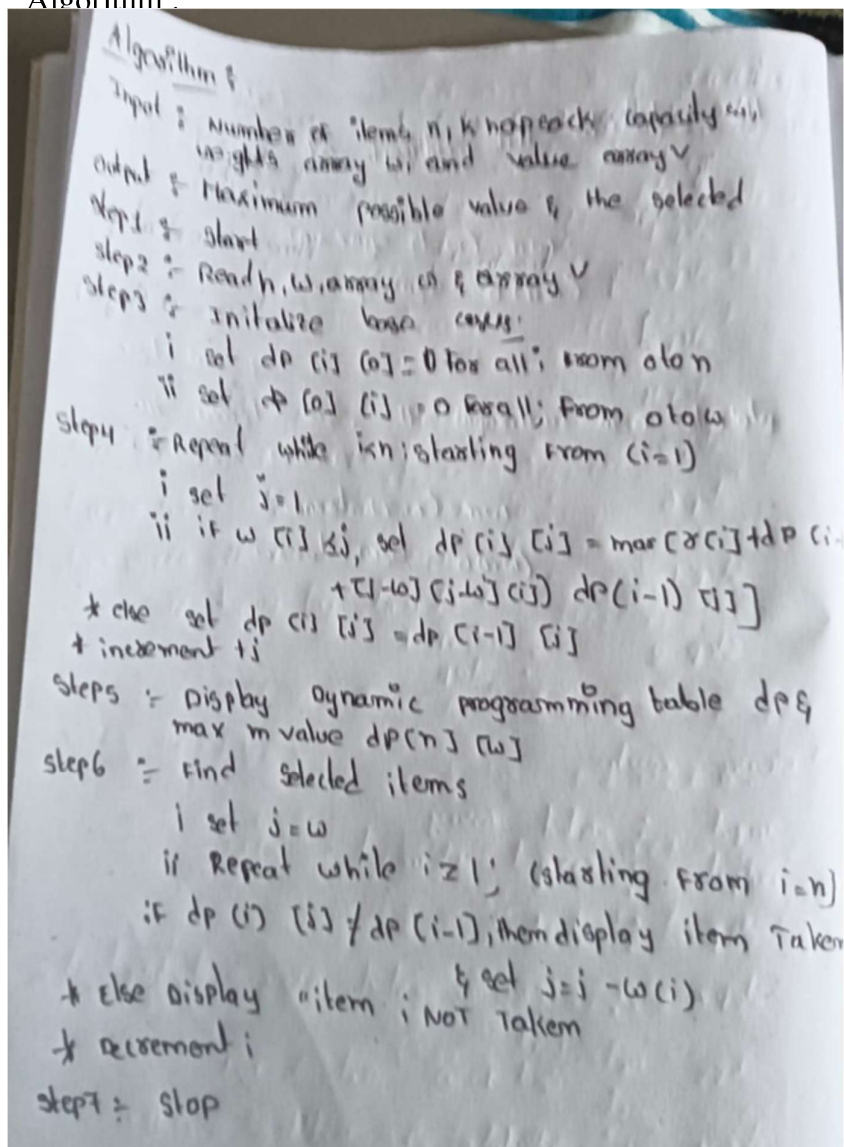


PRACTICAL-5

Aim : To find the maximum value of items that can be placed in a knapsack of capacity W without exceeding its weight limits, and to identify which items are selected, using Dynamic Programming.

Algorithm :



Algorithm :

Input : Number of items n , knapsack capacity w , weights array w , and value array v

Output : Maximum possible value & the selected

Step 1 : Start

Step 2 : Read n, w , array w & array v

Step 3 : Initialize base cases

- i set $dp[i][0] = 0$ for all i from 0 to n
- ii set $dp[0][j] = 0$ for all j from 0 to w

Step 4 : Repeat while $i \leq n$ starting from $i=1$

- i set $j=1$
- ii if $w[i] \leq j$, set $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])$
- * else set $dp[i][j] = dp[i-1][j]$
- * increment j

Step 5 : Display dynamic programming table dp & max value $dp[n][w]$

Step 6 : Find selected items

- i set $j=w$
- if Repeat while $i \geq 1$ (starting from $i=n$)
- if $dp[i][j] \neq dp[i-1][j]$, then display item Taken & set $j = j - w[i]$
- * else display "item i NOT Taken"
- * decrement i

Step 7 : Stop

Program / Code :

```
#include <iostream>
#include <vector>

#include
<algorithm> using
namespace std; class
Knapsack {

private:
    vector<int> w; // weights vector<int> v; // values
    int n, W; // number of items and capacity
    vector<vector<int>> dp; // DP table

public:

    // Get Input void
    getInput() {

        cout << "Enter number of items : "; cin
        >> n;

        cout << "Enter knapsack capacity : ";
        cin >> W;

        w.resize(n + 1);
        v.resize(n + 1);

        // Creating DP table
        // Time Complexity = O(nW) dp.assign(n
        + 1, vector<int>(W + 1, 0));

        for (int i = 1; i <= n; i++) { cout
            << "Item " << i
            << " weight and value : "; cin
            >> w[i] >> v[i];
```

```
}  
}
```

```
// Solve Knapsack using Dynamic Programming  
void solve() {
```

```
    // Time Complexity = O(nW)  
    for (int i = 1; i <= n; i++) {
```

NAME: Raja kiran
ENROLLMENT NO: 92400118654

Page 2 of 7



```
    for (int j = 1; j <= W; j++) {
```

```
        if (w[i] <= j) {
```

```
            dp[i][j] = max( v[i] + dp[i  
                - 1][j - w[i]], dp[i -  
                1][j]
```

FACULTY OF TECHNOLOGY
Department of CE – AI ML DS
01AI0506-DESIGN AND
ANALYSIS OF ALGORITHMS

```

        );

        } else {
dp[i][j] = dp[i - 1][j];
        }
    }
}

// Print DP Table void
printTable() { cout <<
"\nDP Table:\n ";

    for (int j = 0; j <= W; j++) { cout
        << j << " ";
    }
cout << endl;

    // Time Complexity =
    O(nW) for (int i = 0; i <= n;
i++) { cout << "i=" << i <<
    " ";

    for (int j = 0; j <= W; j++) {
        cout << dp[i][j] << " ";
    }

    cout << endl;
}
}

// Find Selected Items
void findItems() {

```



Marwadi
University
Marwadi Chandarana Group



FACULTY OF TECHNOLOGY
Department of CE – AI ML DS
01AI0506-DESIGN AND
ANALYSIS OF ALGORITHMS

```

cout << "\nSelected Items:\n";

int j = W;

// Time Complexity = O(n)
for (int i = n; i >= 1; i--) { if
(dp[i][j] != dp[i - 1][j]) {

    cout << "Item " << i
        << " (w=" << w[i]
        << ", v=" << v[i]
        << ") TAKEN\n";
j -= w[i];

    } else {

        cout << "Item " << i
            << " (w=" << w[i]
            << ", v=" << v[i]
            << ") NOT taken\n";
    }
}
}

// Show Result void
showResult() {

    cout << "\nMaximum Value = "
        << dp[n][W] << endl;
} }; int

main() {

Knapsack k;

k.getInput();
k.solve();
k.printTable();
k.showResult();
k.findItems();

```

NAME: Raja kiran
ENROLLMENT NO: 92400118654


```
    return 0;  
}
```

Output :

```
Enter number of items : 3  
Enter knapsack capacity : 5  
Item 1 weight and value : 2  
1  
Item 2 weight and value : 6  
3  
Item 3 weight and value : 1  
4
```

DP Table:

	0	1	2	3	4	5
i=0	0	0	0	0	0	0
i=1	0	0	1	1	1	1
i=2	0	0	1	1	1	1
i=3	0	4	4	5	5	5

Maximum Value = 5

Selected Items:

Item 3 (w=1, v=4) TAKEN
Item 2 (w=6, v=3) NOT taken
Item 1 (w=2, v=1) TAKEN

Time Complexity :

NAME: Raja kiran
ENROLLMENT NO: 92400118654

Page 5 of 7

Time complexity :-
 n = number of items
 w = knapsack capacity
 the algorithm has two nested loops
 for $i = 1$ to n
 for $j = 1$ to w
 if $w[i] \leq j$
 $K[i][j] = \max \{ P(i) + K(i-1)(j-w[i]), K(i-1)[j] \}$
 else
 $K[i][j] = K(i-1)[j]$
 the outer loop executes n times and the inner loop executes w times for every item.
 $T(n, w) = n \times w$
 $T(n, w) = O(nw)$

1. BEST CASE:

Best case :-
Even if the condition $w[i] \leq j$ is false frequently, the loops still visit every DP table cell
 $T_{best}(h, w) = nw$
Best case = $O(nw)$

NAME: Raja kiran
ENROLLMENT NO: 92400118654

Page 6 of 7

2. AVERAGE CASE:

Average case :-
On average the algorithm still calculates all $n \times w$ table entries
 $T_{average}(h, w) = nw$
Average case = $O(nw)$

3. WORST CASE:

Worst case :-
In worst case, the algorithm also calculates all $n \times w$ entries
 $T_{worst}(h, w) = nw$
Worst case = $O(nw)$

Conclusion :

The 0/1 Knapsack problem was successfully solved using Dynamic Programming. The algorithm finds the maximum possible profit by systematically filling a DP table. It has a fixed time complexity of $O(n \times W)$ for all cases, since the program evaluates each entry of the table to determine the best possible solution.